

OpenClaw Natural Interaction + Basic Control

OpenClaw Natural Interaction + Basic Control

1. Course Content
 - Course Overview
2. Preparation
3. Case Demonstration
 - 3.1 Start OpenClaw Interaction
 - 3.2 Chassis Movement Control
 - 3.3 Chassis In-place Rotation
4. Source Code Analysis
 - 4.1 `move_by_odom()`
 - 4.2 `_odomLinearMove`
 - 4.3 `_odomRotate`
5. Parameter Debugging

1. Course Content

Course Overview

- Use OpenClaw conversation to implement the basic functions of robot motion control

2. Preparation

```
ros2 launch microros_v2_bringup openclaw_service.launch.py
```

```
mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] mcp_service-7] [07/24/26 11:10:11] INFO Starting MCP server transport.py:361 'ROS-MCP-Service' with transport 'streamable-http' on http://0.0.0.0:8000/mcp Started server process [183409] Waiting for application startup. Application startup complete. Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

3. Case Demonstration

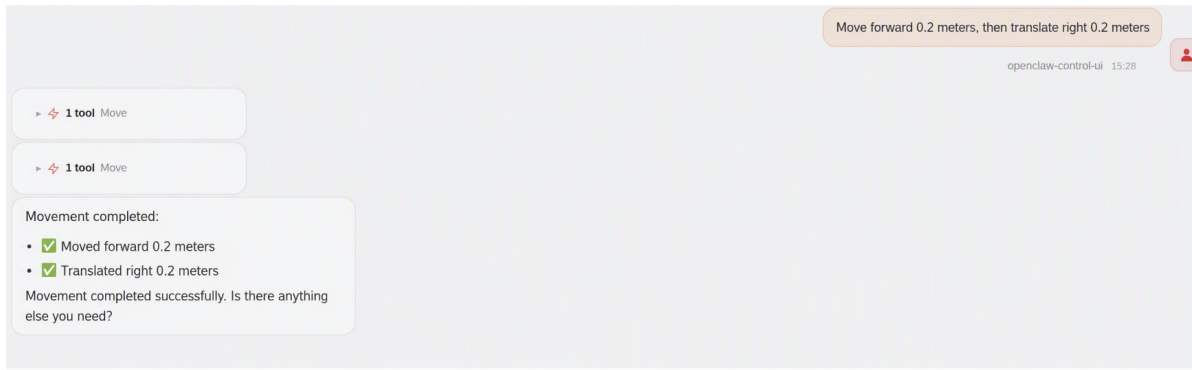
3.1 Start OpenClaw Interaction

[!TIP]

- Use any interaction method to have a conversation with OpenClaw. Here we demonstrate using Web Chat
- You can write the conversation statements according to your own needs. The following is a case demonstration

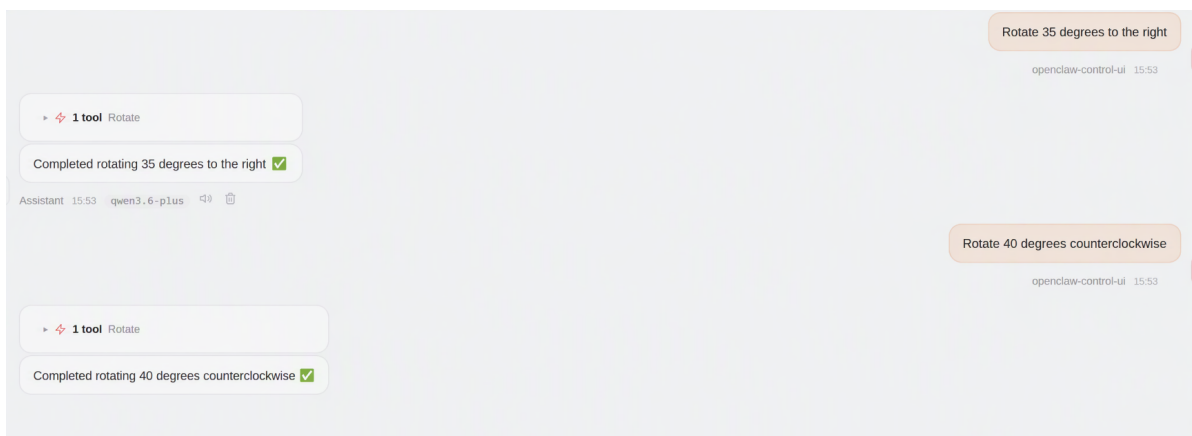
3.2 Chassis Movement Control

- Move forward 0.2 m, then translate 0.2 m to the right



3.3 Chassis In-place Rotation

- Rotate 35 degrees to the right, rotate 40 degrees counterclockwise



4. Source Code Analysis

- Source Code Path

```
$HOME/microros_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

```
$HOME/microros_ws/src/microros_v2_bringup/microros_v2_bringup/base_driver.py
```

4.1 `move_by_odom()`

Code Description:

`move_by_odom()` is the chassis movement control entry function encapsulated in `action_controller.py`. It is responsible for sending a movement goal to the ROS2 Action server and waiting for the execution result.

1. **Function signature:** Accepts three parameters — `distance` (movement distance, meters), `angle` (rotation angle, degrees), `direction` (movement direction) — and returns a `[bool, str]` list representing the execution result and message
2. **Wait for server:** Uses `wait_for_server()` to wait for the MoveByOdom Action server to be ready; returns failure if it times out after 10 seconds
3. **Construct and send goal:** Creates a `MoveByOdom.Goal()` message, fills in the distance, angle, and direction fields, and calls `send_goal_async()` to send asynchronously
4. **Wait for execution result:** First waits for goal confirmation (`send_goal_future`), then waits for execution completion (`result_future`), polling at 0.5s intervals
5. **Return result:** Determines success or failure based on `result.success` and returns the corresponding result message

```
def move_by_odom(self, distance: float = 0.0, angle: float = 0.0, direction:
str = 'forward') -> list[bool, str]:
    ''' Move the chassis by requesting the MoveByOdom action service.
    Supports: forward, backward, turn_left, turn_right.'''
    if not self.move_by_odom_client.wait_for_server(timeout_sec=10.0):
        return [False, "MoveByOdom action server not available"]
    goal = MoveByOdom.Goal()
    goal.distance = float(distance)
    goal.angle = float(angle)
    goal.direction = direction
    send_goal_future = self.move_by_odom_client.send_goal_async(goal)
    while rclpy.ok() and not send_goal_future.done():
        time.sleep(0.5)

    if not send_goal_future.done():
        return [False, "Failed to send MoveByOdom goal"]

    goal_handle = send_goal_future.result()
    result_future = goal_handle.get_result_async()
    while rclpy.ok() and not result_future.done():
        time.sleep(0.5)

    if not result_future.done():
        return [False, "MoveByOdom failed to get result"]

    result = result_future.result().result
    if result.success:
        self.get_logger().info(f"MoveByOdom succeeded: {result.message}")
        return [True, result.message]
    else:
        self.get_logger().warn(f"MoveByOdom failed: {result.message}")
        return [False, result.message]
```

4.2 `_odomLinearMove`

Code Description:

`_odomLinearMove()` is the linear movement control implementation based on odometry feedback in `base_driver.py`. It runs as the callback function of the MoveByOdom Action server.

1. **Parameter parsing:** Gets the target `distance` (distance) and `direction` (direction: forward/backward) from `goal_handle.request`
2. **Speed planning (Ramp-down):** Uses a staged deceleration strategy, gradually reducing the speed when approaching the target to avoid overshoot:
 - Distance > `RAMP_THRESHOLD` (0.3m): drive at the maximum linear speed `odom_linear_speed`
 - `CLOSE_THRESHOLD` (0.1m) < distance ≤ `RAMP_THRESHOLD`: linear interpolation deceleration
 - Distance ≤ `CLOSE_THRESHOLD`: creep at the minimum speed `min_linear_speed`
3. **Odometry feedback:** Each loop calculates the moved distance `moved = |odom_x - start_x|` and reports it in real time via `publish_feedback()`
4. **Completion check:** When `moved ≥ distance - odom_position_tolerance`, the target is considered reached, and `goal_handle.succeed()` is called to return a success result
5. **Exception handling:** Supports timeout detection (`odom_timeout`) and cancel requests (`is_cancel_requested`), returning the corresponding failure result on exception
6. **Velocity publishing:** Determines the sign of the velocity according to direction, and calls `_publishCmdvel(vx, 0.0)` to publish the linear velocity command (vy=0, differential drive does not support lateral movement)

```
def _odomLinearMove(self, goal_handle):
    goal = goal_handle.request
    distance = goal.distance
    direction = goal.direction

    linear_speed = abs(self.odom_linear_speed)
    start_x = self.odom_x

    CLOSE_THRESHOLD = 0.1
    RAMP_THRESHOLD = 0.3
    MIN_SPEED = self.min_linear_speed

    start_time = time.time()
    while rclpy.ok():
        # Timeout check
        if time.time() - start_time > self.odom_timeout:
            self.get_logger().warn(f"Linear movement timeout
({self.odom_timeout}s)")
            goal_handle.abort()
            result = MoveByOdom.Result()
            result.success = False
            result.message = f"Movement timeout after {self.odom_timeout}s"
            return result

        # Cancel check
        if goal_handle.is_cancel_requested:
            self.get_logger().info("Linear movement cancelled")
            goal_handle.canceled()
            result = MoveByOdom.Result()
            result.success = False
            result.message = "Movement cancelled"
            return result

    moved = abs(self.odom_x - start_x)
```

```

# Publish feedback
feedback = MoveByOdom.Feedback()
feedback.moved_distance = float(moved)
feedback.rotated_angle = 0.0
goal_handle.publish_feedback(feedback)

# Check if target distance reached
if moved >= abs(distance) - self.odom_position_tolerance:
    self.get_logger().info(f"Linear movement complete! Moved
{moved:.3f}m")
    goal_handle.succeed()
    result = MoveByOdom.Result()
    result.success = True
    result.message = f"Moved {direction} {moved:.3f}m"
    return result

# Ramp-down: gradual deceleration when approaching target
distance_to_go = abs(distance) - moved
if distance_to_go < CLOSE_THRESHOLD:
    ramp_speed = MIN_SPEED
elif distance_to_go < RAMP_THRESHOLD:
    ramp_speed = MIN_SPEED + (linear_speed - MIN_SPEED) * (
        (distance_to_go - CLOSE_THRESHOLD) / (RAMP_THRESHOLD -
CLOSE_THRESHOLD))
else:
    ramp_speed = linear_speed

# Map direction to velocity (vy=0 for differential drive)
if direction == 'forward':
    vx = ramp_speed
else: # backward
    vx = -ramp_speed

self._publishCmdVel(vx, 0.0)
time.sleep(0.1)

# Guard: should not reach here
goal_handle.abort()
result = MoveByOdom.Result()
result.success = False
result.message = "Unexpected exit from linear control loop"
return result

```

4.3 `_odomRotate`

Code Description:

`_odomRotate()` is the in-place rotation control implementation based on odometry feedback in `base_driver.py`, also running as the callback function of the MoveByOdom Action server.

1. **Direction parsing:** Determines the rotation direction sign `sign` based on `direction` (turn_left/turn_right) (turn left = +1, turn right = -1); aborts and exits on an unknown direction

2. **Angle conversion:** Converts the target angle from degrees to radians `target_angle_rad` and obtains the current maximum angular speed `odom_angular_speed`
3. **Staged deceleration strategy:** Similar to linear movement, uses a three-stage deceleration:
 - Remaining angle > `RAMP_THRESHOLD` ($0.3\text{rad} \approx 17^\circ$): rotate at full speed
 - `CLOSE_THRESHOLD` ($0.1\text{rad} \approx 5.7^\circ$) < remaining angle $\leq$ `RAMP_THRESHOLD`: linear interpolation deceleration
 - Remaining angle $\leq$ `CLOSE_THRESHOLD`: creep at `min_angular_speed`
4. **Odometry feedback:** Each loop calculates the rotated angle `rotated = odom_th - start_th` and reports it in real time via feedback
5. **Completion check:** When `|rotated| >= target_angle_rad - odom_angle_tolerance`, the target angle is considered reached
6. **Velocity publishing:** Calls `_publishcmdvel(0.0, angular_cmd)` to publish the angular velocity command ($v_x=0$, pure rotation without translation)

```
def _odomRotate(self, goal_handle):
    goal = goal_handle.request
    angle_deg = goal.angle
    direction = goal.direction

    if direction == 'turn_left':
        sign = 1.0
    elif direction == 'turn_right':
        sign = -1.0
    else:
        self.get_logger().error(f"Unknown rotation direction: {direction}")
        goal_handle.abort()
        result = MoveByOdom.Result()
        result.success = False
        result.message = f"Unknown rotation direction: {direction}"
        return result

    target_angle_rad = math.radians(abs(angle_deg))
    angular_speed = abs(self.odom_angular_speed)

    # Ramp-down thresholds (radians)
    CLOSE_THRESHOLD = 0.1 # ~5.73°
    RAMP_THRESHOLD = 0.3 # ~17.19°
    MIN_SPEED = self.min_angular_speed

    start_th = self.odom_th

    start_time = time.time()
    while rclpy.ok():
        # Timeout check
        if time.time() - start_time > self.odom_timeout:
            self.get_logger().warn(f"Rotation timeout ({self.odom_timeout}s)")
            goal_handle.abort()
            result = MoveByOdom.Result()
            result.success = False
            result.message = f"Rotation timeout after {self.odom_timeout}s"
            return result

        # Cancel check
```

```

if goal_handle.is_cancel_requested:
    self.get_logger().info("Rotation cancelled")
    goal_handle.canceled()
    result = MoveByOdom.Result()
    result.success = False
    result.message = "Rotation cancelled"
    return result

# Compute angle rotated so far from /odom pose
rotated = self.odom_th - start_th

# Publish feedback
feedback = MoveByOdom.Feedback()
feedback.moved_distance = 0.0
feedback.rotated_angle = float(math.degrees(abs(rotated)))
goal_handle.publish_feedback(feedback)

# Check if target angle reached
if abs(rotated) >= target_angle_rad - self.odom_angle_tolerance:
    rotated_deg = math.degrees(abs(rotated))
    self.get_logger().info(f"Rotation complete! Rotated
{rotated_deg:.1f}°")
    goal_handle.succeed()
    result = MoveByOdom.Result()
    result.success = True
    result.message = f"Rotated {direction} {abs(angle_deg):.1f}°"
    return result

# Ramp-down: gradual deceleration when approaching target
angle_to_go = target_angle_rad - abs(rotated)
if angle_to_go < CLOSE_THRESHOLD:
    ramp_angular = MIN_SPEED
elif angle_to_go < RAMP_THRESHOLD:
    ramp_angular = MIN_SPEED + (angular_speed - MIN_SPEED) * (
        (angle_to_go - CLOSE_THRESHOLD) / (RAMP_THRESHOLD -
CLOSE_THRESHOLD))
else:
    ramp_angular = angular_speed

angular_cmd = sign * ramp_angular
self._publishCmdVel(0.0, angular_cmd)
time.sleep(0.1)

# Guard: should not reach here
goal_handle.abort()
result = MoveByOdom.Result()
result.success = False
result.message = "Unexpected exit from rotation control loop"
return result

```

5. Parameter Debugging

- Parameter file path

```
$HOME/microros_ws/src/microros_v2_bringup/config/common.yaml
```

The following describes the parameters related to chassis odometry control in `common.yaml`:

Parameter	Default	Description
<code>odom_linear_speed</code>	0.1	Maximum linear speed (m/s), the target speed for linear movement
<code>odom_angular_speed</code>	0.5	Maximum angular speed (rad/s), the target angular speed for in-place rotation
<code>odom_position_tolerance</code>	0.02	Linear movement position tolerance (m); considered complete when the difference between the actual and target distance is less than this value
<code>odom_angle_tolerance</code>	0.05	Rotation angle tolerance (rad); considered complete when the difference between the actual and target angle is less than this value
<code>odom_timeout</code>	30.0	Action timeout (s); aborts and returns failure if not completed within this time
<code>min_linear_speed</code>	0.03	Minimum linear speed (m/s), the creep speed when approaching the target, preventing stagnation caused by too low a speed
<code>min_angular_speed</code>	0.2	Minimum angular speed (rad/s), the creep angular speed when approaching the target angle

[!TIP]

- If the robot's movement distance deviates significantly, you can appropriately increase `odom_position_tolerance`
- If the robot rotates imprecisely or overshoots, adjust `odom_angle_tolerance` or reduce `odom_angular_speed`
- If the robot moves too slowly, increase `odom_linear_speed`; conversely, decrease it if oscillation occurs

base_driver:

ros__parameters:

```
odom_linear_speed: 0.15      # Max linear speed (m/s)
odom_angular_speed: 0.5      # Max angular speed (rad/s)
odom_position_tolerance: 0.02 # Linear movement position tolerance (m)
odom_angle_tolerance: 0.05   # Rotation angle tolerance (rad)
odom_timeout: 30.0           # Action timeout (s)
min_linear_speed: 0.03       # Unified min linear speed (m/s)
min_angular_speed: 0.2       # Unified min angular speed (rad/s)
```